



Distributed Systems

Client-Server Computing

TCP Server

Ref: Unix Network Programming, The Socket Networking API by W Richard Stevens



Recap



Socket API functions

```
int socket(int family, int type, int protocol);  
int connect(int sockfd, struct sockaddr *servaddr, int addrlen);  
int bind(int sockfd, const struct sockaddr *myaddr, int addrlen);  
int listen(int sockfd, int backlog);  
int accept(int sockfd, struct sockaddr *peer, int *addrlen);  
int close(int sockfd);
```

```

int
main(int argc, char **argv)
{
    int      sockfd, fd;

    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    printf("%d\n", sockfd);
    fd = open("data", O_RDONLY);
    printf("%d\n", fd);

    for(;;);
    exit(0);
}

```

• ^{TCP} Concurrent server

```

pid_t pid;
int listenfd, connfd;

listenfd = socket(...);
bind(listenfd, ...);
listen(listenfd, 5);

for (;;) {
    connfd = accept(listenfd, ...);
    if (pid = fork()) == 0 {
        close(listenfd);
        do(connfd);
        close(connfd);
        exit(0);
    }
    close(connfd);
}

```

• ^{TCP} Iterative server

```

int listenfd, connfd;

listenfd = socket(...);
bind(listenfd, ...);
listen(listenfd, 5);

for (;;) {
    connfd = accept(listenfd, ...);
    do(connfd);
    close(connfd);
}

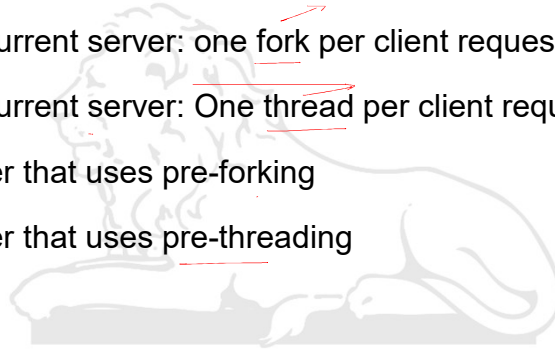
```

→ Lightly loaded server
No overhead of creating a new process

TCP Server design alternatives



- Iterative server
- Concurrent server: one fork per client request ←
- Concurrent server: One thread per client request
- Server that uses pre-forking
- Server that uses pre-threading



IIT ROORKEE ■ ■ ■

TCP Echo Server

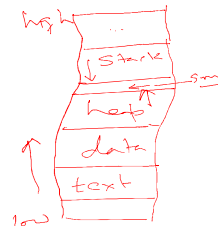
```
int main(int argc, char **argv)
{
    ...
    listenfd = socket(AF_INET, SOCK_STREAM, 0);

    bzero(&servaddr, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
    servaddr.sin_port = htons(SERV_PORT);

    bind(listenfd, (struct sockaddr *) &servaddr, sizeof(servaddr));
    listen(listenfd, 5);

    for ( ; ; ) {
        cllen = sizeof(cliaddr);
        connfd = accept(listenfd, (struct sockaddr*) &cliaddr, &cllen);

        if ( (childpid = fork()) == 0 ) { /* child process */
            close(listenfd); /* close listening socket */
            n = read(connfd, line, 512);
            → write(connfd, line, n);
            close(connfd);
            exit(0);
        }
        close(connfd);
    }
}
```



TCP Echo Client

```

int main(int argc, char **argv)
{
    int                sockfd;
    struct sockaddr_in  servaddr;

    if (argc != 2)
        err_quit("usage: tcpcli <IPaddress>");

    sockfd = socket(AF_INET, SOCK_STREAM, 0);

    bzero(&servaddr, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = inet_addr(SERV_HOST_ADDR);
    servaddr.sin_port = htons(SERV_PORT);

    connect(sockfd, (struct sockaddr*) &servaddr, sizeof(servaddr));

    fgets(sendline, 512, stdin);          /* do it all */
    n = strlen(sendline);
    → write(sockfd, sendline, n);
    → n = read(sockfd, recvline, 512);
    if (n < 0) ...
    recvline[n] = 0;
    fputs(recvline, stdout);
    exit(0);
}

```

- Socket descriptor is always passed as an argument
- Socket address structure is always passed as reference to any of the socket functions
- Length of the structure is also passed (How?)
 - With **bind** and **connect** the length is passed by value
 - With **accept** a pointer to length is passed
- What if the **connect** fails?
- When child closes connected socket but parent does not closes the connected socket?

TCP Echo server and client

Normal Startup

- server blocks in accept
- before starting the client

$$\frac{0}{1} \text{ nat slot} - a$$

```
Proto Recv-Q Send-Q Local Address      Foreign Address    State
tcp        0      0 *:5000            *:*
```

- client starts
- when connect returns in client and accept returns in the server

```
Proto Recv-Q Send-Q Local Address Foreign Address State
tcp 0 0 *:5000 *: * LISTEN
tcp 0 0 localhost:32776 localhost:5000 ESTABLISHED
tcp 0 0 localhost:5000 localhost:32776 ESTABLISHED
```

- output of ps?

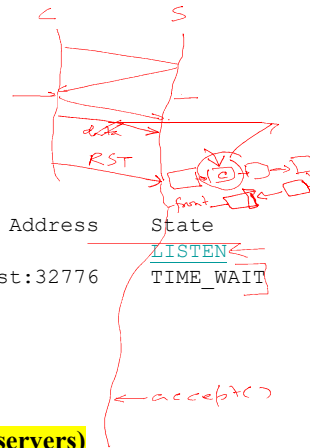
- Normal Termination of client / server

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	State
tcp	0	0	*:5000	*:*	<u>LISTEN</u>
tcp	0	0	localhost:5000	localhost:32776	TIME WAIT

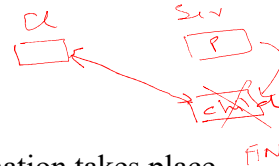
- Output of **ps** (Normal Termination)

Connection aborts before accept returns (on busy servers)

- o Three-way handshake completes and then the client TCP sends an RST
- o On the server side the connection is queued by its TCP waiting for the server process to call accept
- Handling - Implementation dependent



Termination of Server child process



- kill the server child process
 - first half of the TCP connection termination takes place
 - client process is blocked in the call to fgets

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	State
tcp	0	0	*:65000	*:*	LISTEN
tcp	1	0	localhost:32778	localhost:65000	CLOSE_WAIT ←
tcp	0	0	localhost:65000	localhost:32778	FIN_WAIT2 ←

- User types a line
 - client sends it to server
 - the server TCP responds with an RST



• Crashing of server host

(different from crashing of server ^{child} process)

(different from shutdown)

- client TCP continually retransmits the data segments
- either the client TCP finally gives up and returns an error or
- some intermediate router says that the host is unreachable

2

Crashing and Rebooting of server host

- client sends a TCP data segment to the server host
- server TCP loses all information about the connections before crash
- server TCP responds to the received data segment by RST
- an error is returned to the client process (ECONNRESET)

Shutdown of server Host

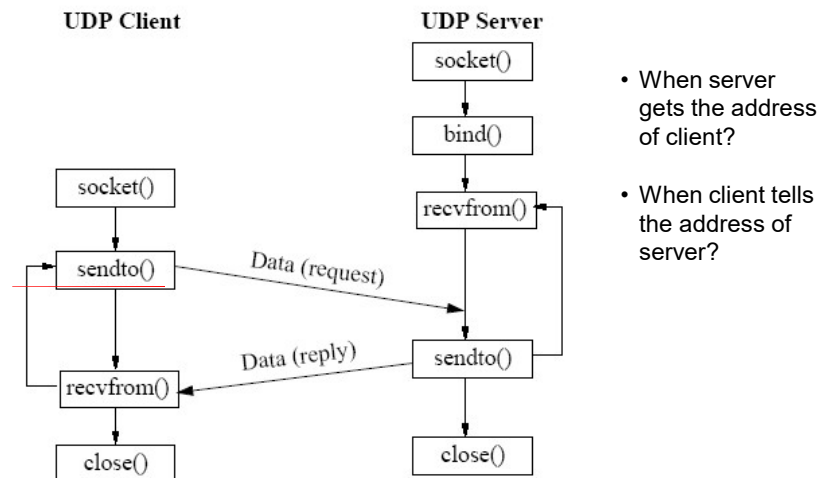
- init first sends SIGTERM and then SIGKILL
- same as termination of server child



UDP Client-Server



Socket System calls for **connection-less** protocols



```
recvfrom(sockfd, mesg, 512, flags, (struct sockaddr *) &cliaddr, &len);
```

```
accept(listenfd, (struct sockaddr *) &cliaddr, &clilen);
```

```
read(connfd, line, 512);
```

```
sendto(sockfd, mesg, n, flags, (struct sockaddr *) &servaddr, len);
```

```
connect(sockfd, (struct sockaddr *) &servaddr, sizeof(servaddr));
```

```
write(connfd, line, n);
```


UDP Echo Server

#define SERV_PORT 5000

int main(int argc, char **argv)

{

```

    int          sockfd, len, n;
    char          msg[512];
    struct sockaddr_in servaddr, cliaddr;

```

sockfd = socket(AF_INET, SOCK_DGRAM, 0);

```

    bzero(&servaddr, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
    servaddr.sin_port = htons(SERV_PORT);

```

bind(sockfd, (struct sockaddr *) &servaddr, sizeof(servaddr));

for (; ;) {

len = sizeof(cliaddr);

n = recvfrom(sockfd, msg, 512, 0, (struct sockaddr *) &cliaddr, &len);

sendto(sockfd, msg, n, 0, (struct sockaddr *) &cliaddr, len);

}

}

UDP Echo Client

#define SERV_PORT 5000

int main(int argc, char **argv)

{

```

    ...
    bzero(&servaddr, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = inet_addr("127.0.0.1");
    servaddr.sin_port = htons(SERV_PORT);
    servlen = sizeof(servaddr);

```

sockfd = socket(AF_INET, SOCK_DGRAM, 0);

while (fgets(sendline, MAXLINE, stdin) != NULL) {

```

    → sendto(sockfd, sendline, strlen(sendline), 0,
      (struct sockaddr *) &servaddr, servlen);

```

```

    → n = recvfrom(sockfd, recvline, MAXLINE, 0, NULL, NULL); //??

```

recvline[n] = 0; /* null terminate */

```

    → fputs(recvline, stdout);

```

}

exit(0);

}